

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

про виконання лабораторної роботи №1
з дисципліни «Методи та технології обчислень»

на тему:

«Інтерполяція кубічними сплайнами»

Виконав:

студент групи ФЕП-31

Дораж Михайло

Перевірив:

доцент

Мисюк Роман Володимирович

Львів — 2026

1. Мета роботи

Вивчити метод інтерполяції кубічними сплайнами та метод прогонки для розв'язування систем лінійних алгебраїчних рівнянь із тридіагональною матрицею; застосувати їх до реальних геодезичних — побудувати гладкий профіль висоти маршруту від станції Заросляк до вершини гори Говерла та проаналізувати складність маршруту.

2. Основні теоретичні відомості

Кубічний сплайн — математична модель абсолютно гнучкого, пружного та тонкого стрижня. Якщо стрижень закріпити в кінцях, він прийме форму, що мінімізує його потенціальну енергію; умова рівноваги $S^{(4)}(x) = 0$ задовольняється алгебраїчним многочленом третього степеня.

Нехай задано вузли $x_0 < x_1 < \dots < x_n$ і значення функції $y_i = f(x_i)$. На кожному відрізку $[x_{i-1}, x_i]$, $i = 1, \dots, n$, сплайн має вигляд:

$$s_i(x) = a_i + b_i(x - x_{i-1}) + c_i(x - x_{i-1})^2 + d_i(x - x_{i-1})^3.$$

Для знаходження $4n$ коефіцієнтів використовують умови: проходження сплайна через усі вузли ($2n$ рівнянь), неперервність першої та другої похідних у внутрішніх вузлах ($2n - 2$ рівнянь) та дві крайові умови. Для вільного (природного) сплайна кривизна в крайніх вузлах дорівнює нулю:

$$S''(x_0) = 0, \quad S''(x_n) = 0 \quad \Rightarrow \quad c_1 = 0, \quad c_{n+1} = 0.$$

Позначивши $h_i = x_i - x_{i-1}$ та виключивши коефіцієнти a_i, b_i, d_i , отримуємо систему рівнянь відносно c_i ($i = 2, \dots, n$):

$$h_{i-1}c_{i-1} + 2(h_{i-1} + h_i)c_i + h_ic_{i+1} = 3\left(\frac{y_i - y_{i-1}}{h_i} - \frac{y_{i-1} - y_{i-2}}{h_{i-1}}\right).$$

Решта коефіцієнтів обчислюється за формулами:

$$a_i = y_{i-1}, \quad b_i = \frac{y_i - y_{i-1}}{h_i} - \frac{h_i}{3}(c_{i+1} + 2c_i), \quad d_i = \frac{c_{i+1} - c_i}{3h_i}.$$

Система для c_i має тридіагональну матрицю $\alpha_i c_{i-1} + \beta_i c_i + \gamma_i c_{i+1} = \delta_i$ і розв'язується **методом прогонки**, що вимагає $O(n)$ операцій. На етапі прямої прогонки розв'язок шукаємо у вигляді $c_i = A_i c_{i+1} + B_i$, де

$$A_1 = -\frac{\gamma_1}{\beta_1}, \quad B_1 = \frac{\delta_1}{\beta_1}; \quad A_i = -\frac{\gamma_i}{\alpha_i A_{i-1} + \beta_i}, \quad B_i = \frac{\delta_i - \alpha_i B_{i-1}}{\alpha_i A_{i-1} + \beta_i}.$$

На етапі зворотної прогонки $c_m = B_m$, далі послідовно $c_i = A_i c_{i+1} + B_i$. Якщо $|\beta_i| \geq |\alpha_i| + |\gamma_i|$ і хоча б для одного i нерівність строга, ділення на нуль не виникає і метод

стійкий щодо похибок округлення. Для перевірки розв'язку обчислюють ліву частину системи й порівнюють її з вектором вільних членів.

3. Завдання

Застосувати чисельні методи до реальних геоданих — GPS-координат маршруту від станції Заросляк до вершини гори Говерла: отримати висоти точок через відкритий API Open-Elevation, побудувати гладкий профіль висоти кубічним сплайном (коефіцієнти c_i — методом прогонки), оцінити вплив кількості вузлів (10, 15, 20) на точність та проаналізувати складність маршруту.

4. Хід роботи

Програму реалізовано мовою Python у середовищі PyCharm (файл *main.py*) з використанням бібліотек *requests*, *NumPy* та *Matplotlib*.

4.1. Отримання даних і табуляція

Виконано GET-запит до Open-Elevation API для 21 точки маршруту. У відповідь отримано широту, довготу та висоту кожної точки. Результати табуляції записано в текстовий файл *tabulation.txt*.

```
response = requests.get(url, timeout=60)
data = response.json()
results = data["results"]
n = len(results) # 21 вузол
with open("tabulation.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(lines) + "\n")
```

4.2. Кумулятивна відстань

Відстань між сусідніми точками обчислено за формулою гаверсинуса ($R = 6371\,000$ м), кумулятивну відстань — як суму відстаней від початку маршруту:

$$d = 2R \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right), \quad a = \sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}.$$

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371000
    phi1, phi2 = np.radians(lat1), np.radians(lat2)
    dphi = np.radians(lat2 - lat1)
    dlamba = np.radians(lon2 - lon1)
    a = np.sin(dphi/2)**2 + np.cos(phi1)*np.cos(phi2)*np.sin(dlamba/2)**2
    return 2*R*np.arctan2(np.sqrt(a), np.sqrt(1-a))

distances = [0.0]
for i in range(1, n):
    distances.append(distances[-1] + haversine(*coords[i-1], *coords[i]))
```

Таблиця 1 — Табуляція вузлів маршруту

№	Широта, °	Довгота, °	Висота, м	Кумулятивна відстань, м
0	48.164214	24.536044	1269.42	0.00
1	48.164983	24.534836	1293.43	123.85
2	48.165605	24.534068	1293.43	213.45
3	48.166228	24.532915	1325.24	323.50
4	48.166777	24.531927	1316.99	418.87
5	48.167326	24.530884	1323.59	517.41
6	48.167011	24.530061	1323.59	587.78
7	48.166053	24.528039	1346.29	771.72
8	48.166655	24.526064	1376.77	932.77
9	48.166497	24.523574	1413.13	1118.27
10	48.166128	24.520214	1482.93	1370.81
11	48.165416	24.517170	1521.99	1610.05
12	48.164546	24.514640	1547.84	1821.16
13	48.163412	24.512980	1625.94	1997.39
14	48.162331	24.511715	1742.89	2149.88
15	48.162015	24.509462	1787.37	2320.63
16	48.162147	24.506932	1822.82	2508.86
17	48.161751	24.504244	1883.61	2713.03
18	48.161197	24.501793	1972.86	2904.98
19	48.160580	24.500537	1972.86	3020.67
20	48.160250	24.500106	2016.72	3069.34

За отриманими даними побудовано графік залежності висоти від кумулятивної відстані (рис. 1).

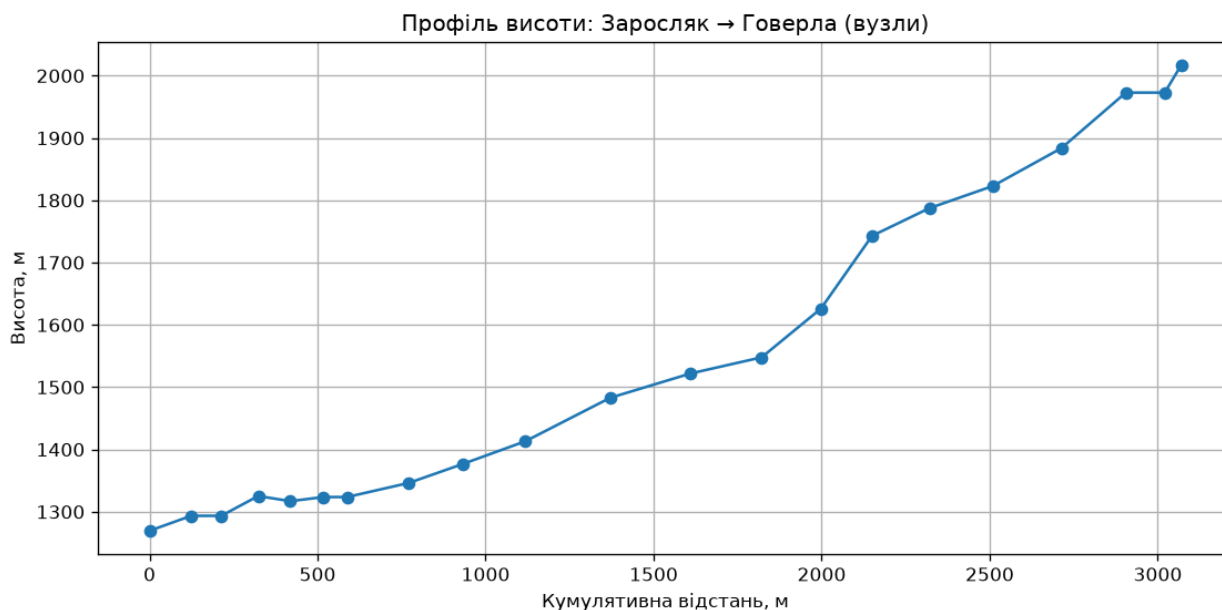


Рисунок 1 — Профіль висоти маршруту Заросляк → Говерла (вузли)

4.3. Формування тридіагональної системи

Функція *build_system* обчислює кроки h_i та коефіцієнти α , β , γ , δ системи для внутрішніх вузлів ($c_0 = c_n = 0$, індексація з нуля):

```
def build_system(x, y):
    h = np.diff(x)
    m = len(x) - 2 # кількість невідомих
    alpha, beta = np.zeros(m), np.zeros(m)
    gamma, delta = np.zeros(m), np.zeros(m)
    for k in range(m):
        i = k + 1 # номер внутрішнього вузла
        alpha[k] = h[i-1] if k > 0 else 0.0
        beta[k] = 2 * (h[i-1] + h[i])
        gamma[k] = h[i] if k < m - 1 else 0.0
        delta[k] = 3 * ((y[i+1] - y[i]) / h[i] - (y[i] - y[i-1]) / h[i-1])
    return h, alpha, beta, gamma, delta
```

Для 21 вузла отримано систему з 19 рівнянь. Її коефіцієнти, а також прогонкові коефіцієнти A_i , B_i наведено в таблиці 2.

Таблиця 2 — Коефіцієнти тридіагональної системи та прогонкові коефіцієнти (21 вузол)

k	α_k	β_k	γ_k	δ_k	A_k	B_k
1	0.0000	426.8953	89.5984	-0.581677	-0.20988373	-0.00136258
2	89.5984	399.2982	110.0507	0.867234	-0.28923194	0.00260010
3	110.0507	410.8434	95.3710	-1.126989	-0.25162971	-0.00372845
4	95.3710	387.8196	98.5388	0.460690	-0.27084391	0.00224362
5	98.5388	337.8215	70.3720	-0.200935	-0.22617972	-0.00135639
6	70.3720	508.6301	183.9431	0.370260	-0.37332677	0.00094520
7	183.9431	689.9750	161.0444	0.197533	-0.25920383	0.00003810
8	161.0444	693.0901	185.5006	0.020308	-0.28479552	0.00002176
9	185.5006	876.0926	252.5456	0.241057	-0.30676187	0.00028790
10	252.5456	983.5645	239.2366	-0.339378	-0.26403094	-0.00045480
11	239.2366	900.6912	211.1090	-0.122450	-0.25206280	-0.00001629
12	211.1090	774.6828	176.2324	0.962204	-0.24426850	0.00133844
13	176.2324	657.4323	152.4837	0.971388	-0.24818952	0.00119715
14	152.4837	646.4849	170.7587	-1.519469	-0.28055781	-0.00279642
15	170.7587	717.9633	188.2229	-0.216448	-0.28090644	0.00038962
16	188.2229	784.7936	204.1739	0.328219	-0.27895632	0.00034824
17	204.1739	792.2400	191.9461	0.501613	-0.26105019	0.00058550
18	191.9461	615.2851	115.6964	-1.394837	-0.20470807	-0.00266681
19	115.6964	328.7261	0.0000	2.704113	0.00000000	0.00987619

Умова діагональної переваги $|\beta_i| \geq |\alpha_i| + |\gamma_i|$ виконується строго, оскільки $2(h_{i-1} + h_i) > h_{i-1} + h_i$, тому метод прогонки стійкий.

4.4. Розв'язування системи методом прогонки

```
def thomas(alpha, beta, gamma, delta):
    m = len(beta)
    A, B = np.zeros(m), np.zeros(m)
    A[0] = -gamma[0] / beta[0]          # пряма прогонка
    B[0] = delta[0] / beta[0]
    for i in range(1, m):
        denom = alpha[i] * A[i-1] + beta[i]
        A[i] = -gamma[i] / denom
        B[i] = (delta[i] - alpha[i] * B[i-1]) / denom
    x = np.zeros(m)
    x[-1] = B[-1]                      # зворотна прогонка
    for i in range(m - 2, -1, -1):
        x[i] = A[i] * x[i+1] + B[i]
    return x
```

Для перевірки правильності розв'язку обчислено вектор лівої частини системи та порівняно його з вектором вільних членів: максимальна нев'язка $\max|Mc - \delta| \approx 3,3 \cdot 10^{-16}$, тобто розв'язок точний у межах похибки округлення.

4.5. Обчислення коефіцієнтів сплайнів

```
def cubic_spline(x, y):
    h, alpha, beta, gamma, delta = build_system(x, y)
    c_full = np.zeros(len(x))          # c_0 = c_n = 0 (вільний сплайн)
    c_full[1:-1] = thomas(alpha, beta, gamma, delta)
    a = y[:-1]
    c = c_full[:-1]
    b = (y[1:] - y[:-1]) / h - h * (c_full[1:] + 2 * c_full[:-1]) / 3
    d = (c_full[1:] - c_full[:-1]) / (3 * h)
    return a, b, c, d

def spline_eval(x_nodes, coeffs, xx):
    a, b, c, d = coeffs
    idx = np.clip(np.searchsorted(x_nodes, xx, side="right") - 1, 0, len(a) - 1)
    t = xx - x_nodes[idx]
    return a[idx] + b[idx]*t + c[idx]*t**2 + d[idx]*t**3
```

Таблиця 3 — Коефіцієнти кубічних сплайнів (21 вузол)

i	Відрізок, м	a_i	b_i	c_i	d_i
1	[0.0; 123.8]	1269.4177	0.28369856	0.0000000000	-0.000005854899
2	[123.8; 213.4]	1293.4312	0.01427993	-0.0021753750	0.000022500383
3	[213.4; 323.5]	1293.4312	0.16635022	0.0038726187	-0.000025055960
4	[323.5; 418.9]	1325.2444	0.10834894	-0.0043996600	0.000024700450
5	[418.9; 517.4]	1316.9867	-0.05685282	0.0026674585	-0.000014317034
6	[517.4; 587.8]	1323.5867	0.05179274	-0.0015648924	0.000011778937
7	[587.8; 771.7]	1323.5867	0.00653904	0.0009218279	-0.000001557049
8	[771.7; 932.8]	1346.2889	0.18761824	0.0000626025	-0.000000325255
9	[932.8; 1118.3]	1376.7689	0.18247507	-0.0000945389	0.000000903668
10	[1118.3; 1370.8]	1413.1333	0.24068803	0.0004083543	-0.000001057243
11	[1370.8; 1610.1]	1482.9334	0.24465349	-0.0003926524	0.000000219156
12	[1610.1; 1821.2]	1521.9911	0.09440942	-0.0002353621	0.000001743911
13	[1821.2; 1997.4]	1547.8400	0.22819805	0.0008691041	0.000001990332
14	[1997.4; 2149.9]	1625.9423	0.71997304	0.0019213871	-0.000010579173
15	[2149.9; 2320.6]	1742.8933	0.56799536	-0.0029180683	0.000006542668
16	[2320.6; 2508.9]	1787.3733	0.14374806	0.0004335844	-0.000001045040
17	[2508.9; 2713.0]	1822.8223	0.19589848	-0.0001565171	0.000003209626
18	[2713.0; 2905.0]	1883.6133	0.53338392	0.0018094486	-0.000011284407
19	[2905.0; 3020.7]	1972.8578	-0.01924782	-0.0046885465	0.000041962514
20	[3020.7; 3069.3]	1972.8578	0.58094354	0.0098761877	-0.000067645145

Сплайн проходить через усі вузли: максимальне відхилення у вузлах становить $\approx 2 \cdot 10^{-13}$ м.

4.6. Сплайни з 10, 15 та 20 вузлами

Для дослідження впливу кількості вузлів на точність з 21 точки рівномірно (за індексами) вибрано 10, 15 та 20 вузлів; перша й остання точки маршруту включені завжди. Для кожного набору обчислено коефіцієнти сплайна (повні таблиці виведено в консолі програми) та побудовано графіки (рис. 2).

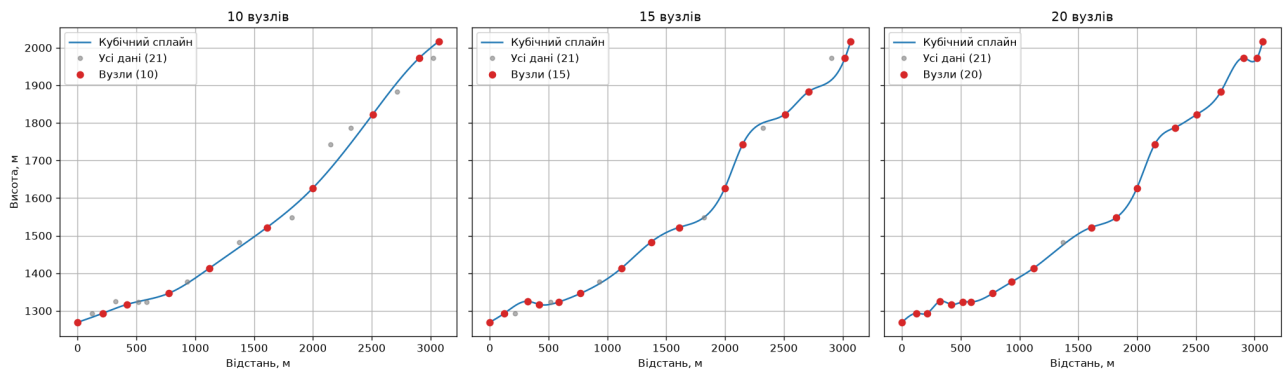


Рисунок 2 — Кубічні сплайни, побудовані за 10, 15 та 20 вузлами

4.7. Оцінка точності

За задану функцію $f(x)$ взято сплайн, побудований за всіма 21 вузлами, за її наближення $\varphi(x)$ — сплайни за 10, 15 та 20 вузлами. Похибку $\varepsilon(x) = |f(x) - \varphi(x)|$ обчислено на всьому відрізку $[x_0; x_{20}]$ (2000 точок). Додатково обчислено похибку в пропущених вузлах, де відоме справжнє значення висоти.

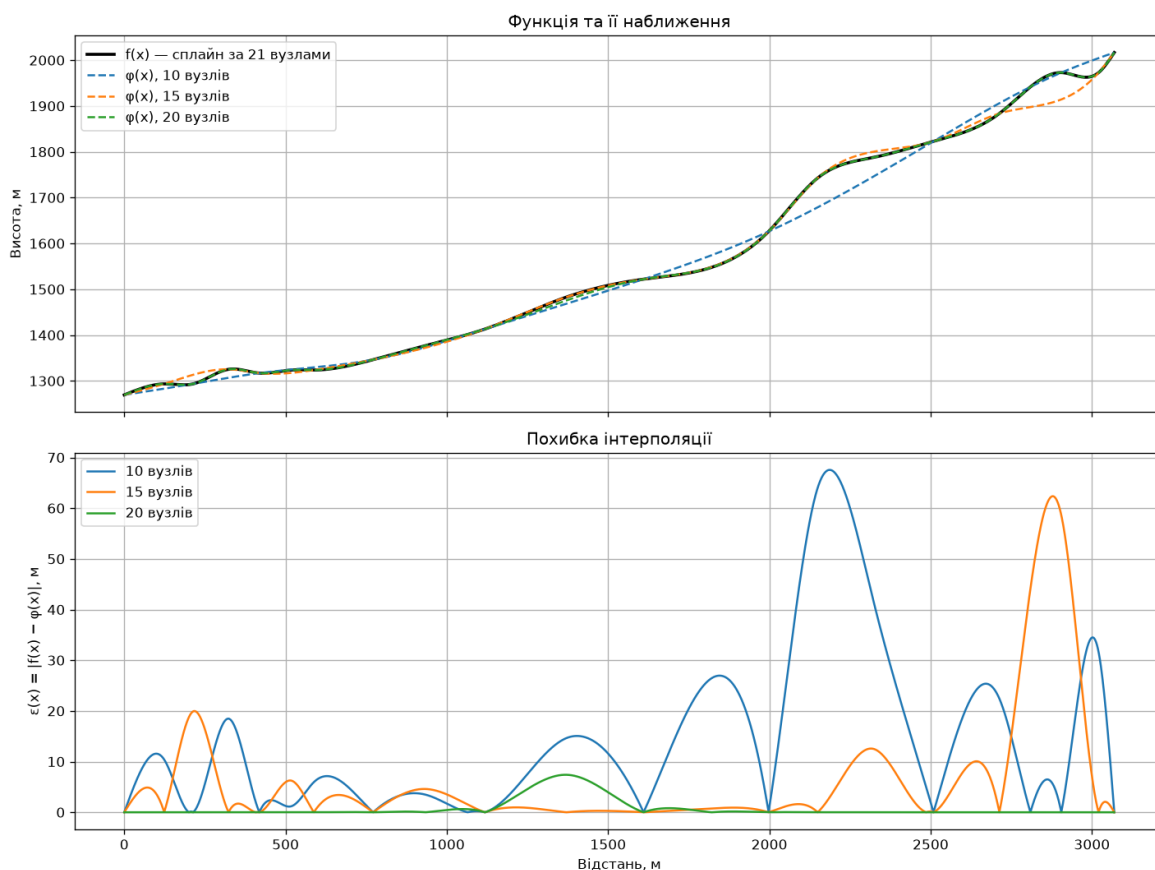


Рисунок 3 — Функція $f(x)$, її наближення $\varphi(x)$ та похибка $\varepsilon(x)$

Таблиця 4 — Вплив кількості вузлів на точність інтерполяції

Кількість вузлів	Пропущені вузли	max $\varepsilon(x)$, м	Середня $\varepsilon(x)$, м	Мах похибка в пропущених вузлах, м
10	1, 3, 5, 6, 8, 10, 12, 14, 15, 17, 19	67.55	14.40	64.04
15	2, 5, 8, 12, 15, 18	62.36	6.40	58.63
20	10	7.40	0.74	7.40

Спостереження:

- зі збільшенням кількості вузлів середня похибка інтерполяції зменшується: 14,40 м $\rightarrow$ 6,40 м $\rightarrow$ 0,74 м;
- при 10 вузлах сплайн згладжує локальні перегини рельєфу; найбільша похибка (≈ 68 м) виникає на крутій ділянці 2000–2500 м, де пропущено два вузли поспіль (14 і 15);

- при 15 вузлах максимальна похибка (≈ 62 м) майже така сама, як при 10, оскільки пропущено вузол 18 — різкий підйом (1884 $\rightarrow$ 1973 м) перед плато під вершиною; поза цією ділянкою похибка невелика;
- при 20 вузлах (пропущено лише вузол 10) сплайн практично збігається з еталоном, похибка не перевищує 7,4 м;
- отже, максимальна похибка залежить не лише від кількості вузлів, а й від їх розташування: вузли потрібні насамперед там, де рельєф різко змінює нахил або кривизну.

4.8. Характеристики маршруту та аналіз градієнта

Додатково обчислено загальні характеристики маршруту, градієнт (через похідну сплайна, у відсотках) та механічну роботу підйому для людини масою 80 кг ($A = mgH$, $g = 9,81$ м/с²).

```
total_ascent = sum(max(elevations[i] - elevations[i-1], 0) for i in range(1, n))
total_descent = sum(max(elevations[i-1] - elevations[i], 0) for i in range(1, n))
grad_full = np.gradient(yy_full, xx) * 100 # градієнт, %
energy = 80 * 9.81 * total_ascent # механічна робота, Дж
```

Таблиця 5 — Характеристики маршруту Заросляк $\rightarrow$ Говерла

Характеристика	Значення
Загальна довжина маршруту	3069.34 м
Сумарний набір висоти	755.56 м
Сумарний спуск	8.26 м
Максимальний підйом	106.14 %
Максимальний спуск	-19.38 %
Середній градієнт (за модулем)	25.81 %
Частка маршруту з крутизною > 15 %	71.4 %
Механічна робота ($m = 80$ кг)	592 967 Дж = 592.97 кДж $\approx$ 141.72 ккал

Таблиця 6 — Ділянки з крутизною понад 15 %

Ділянка, м	Довжина, м	Макс. крутизна, %
0.0 – 86.0	86.0	28.4
211.9 – 317.8	105.9	36.6
377.7 – 388.5	10.7	-15.3
695.6 – 1506.3	810.7	29.3
1768.8 – 2269.4	500.6	83.6
2329.3 – 2885.1	555.8	63.0
2925.0 – 2960.3	35.3	-19.4
2995.6 – 3069.3	73.7	106.1

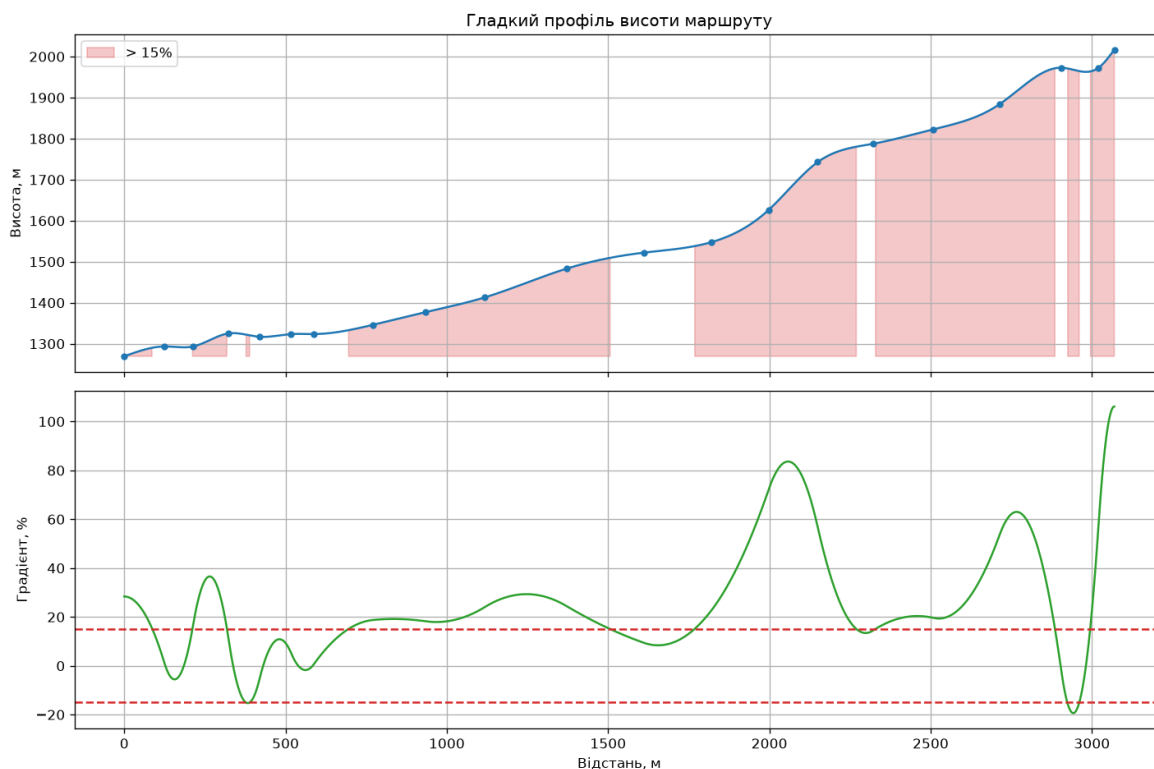


Рисунок 4 — Гладкий профіль висоти маршруту (ділянки з крутизною > 15 % виділено) та градiєнт

Пікове значення градiєнта $\approx 106\%$ на останніх ~ 50 м пояснюється різкою зміною висоти між близько розташованими вузлами ($+44$ м на 49 м) і коливанням сплайна, тому значення ухилу на коротких ділянках слід інтерпретувати обережно. Загалом маршрут складний: середній градiєнт $\approx 26\%$, а понад 70% шляху має крутизну більше 15% .

5. Висновки

У ході виконання лабораторної роботи вивчено метод інтерполяції кубічними сплайнами та метод прогонки для розв'язування систем лінійних алгебраїчних рівнянь із тридіагональною матрицею. Обидва методи реалізовано програмно мовою Python і застосовано до реальних геоданих: за висотами, отриманими з Open-Elevation API, побудовано гладкий профіль маршруту від станції Заросляк до вершини Говерли.

Система рівнянь для коефіцієнтів c_i має діагональну перевагу, тому метод прогонки стійкий; нев'язка розв'язку становить близько 10^{-16} , а сплайн проходить через усі вузли з точністю до похибки округлення.

Дослідження впливу кількості вузлів показало, що середня похибка зменшується зі збільшенням кількості вузлів ($14,4$ м $\rightarrow$ $6,4$ м $\rightarrow$ $0,7$ м для 10, 15 і 20 вузлів), а максимальна похибка суттєво залежить від того, чи потрапили у вибірку точки в місцях різкої зміни рельєфу. Отже, для реальних геоданих точність

сплайн-інтерполяції визначається щільністю та розташуванням вузлів.

Аналіз маршруту показав: довжина $\approx 3,07$ км, набір висоти ≈ 756 м, середній градієнт ≈ 26 %, понад 70 % шляху має крутизну більше 15 %; механічна робота підйому для людини масою 80 кг ≈ 593 кДж (≈ 142 ккал). Маршрут можна оцінити як складний.